

Doc. no.: P0186R0
Date: 2016-02-11
Reply to: Beman Dawes <bdawes at acm dot org>
Eric Niebler <eric dot niebler at gmail dot com>
Casey Carter <casey at carter dot net>
Audience: Library Evolution

Iterator Facade Library Proposal for Ranges

"We are what we pretend to be, so we must be careful about what we pretend to be." - Kurt Vonnegut

Summary

Proposes a library component for easily creating conforming iterators. Based on existing practice. Depends only on the C++17 working paper plus Concepts TS and Ranges TS. Breaks no existing code or ABI's. Two open-source implementations including test suites available. Proposed wording provided.

Table of Contents

[Iterator Facade Library Proposal for Ranges](#)

[Summary](#)

[Table of Contents](#)

[Introduction](#)

[Problem](#)

[Solution](#)

[Proposal](#)

[History](#)

[Example](#)

[Design Decisions](#)

[Use Boost iterator facade as a role model](#)

[Base design on Concepts and Ranges](#)

[Use Cursor mixins to supply details](#)

[Proposed wording](#)

[24.8.9 Basic Iterators](#)

[24.8.9.1 Cursors](#)

[24.8.9.1.1 Namespace cursor synopsis](#)

[24.8.9.1.2 Cursor traits](#)

[24.8.9.1.2.1 single pass](#)

[24.8.9.1.2.2 contiguous](#)

[24.8.9.1.2.3 Types](#)

[24.8.9.1.3 Concepts](#)

[24.8.9.2 Class template basic mixin](#)

[24.8.9.2.1 Constructors](#)

[24.8.9.2.2 T object access](#)

[24.8.9.3 Class template basic iterator](#)

[24.8.9.3.1 Synopsis](#)

[24.8.9.3.2 Requirements](#)

[24.8.9.3.3 Constructors, assignments, and moves](#)

[24.8.9.3.4 Dereferences](#)

[24.8.9.3.5 Modifiers](#)

[24.8.9.3.6 basic iterator nonmember functions](#)

[Open questions](#)

[Acknowledgements](#)

[References](#)

Introduction

Problem

Iterators that conform to the requirements of the C++ standard library are tedious to write and difficult to write correctly. They are tedious to write because although they need only a few core functions, they also need subsidiary types, functions, and other boilerplate. Conforming iterators are difficult to write correctly because each iterator category has a subtly differing set of requirements, making it all too easy to get subsidiary types, functions, or other boilerplate wrong.

Solution

A class template that is given a few implementation functions can generate the facade (i.e. public interface and private implementation) for a fully conforming iterator, including all boilerplate. Boost `iterator_facade` pioneered this approach and has been in wide use since 2001^[1]. It eases writing conforming iterators and has proven less error prone than hand-coded iterators. The generated iterator conforms to an extended set of requirements based on the C++98 iterator requirements. Others, such as Chandler Carruth's LLVM `iterator_facade_base`^[2], have provided similar classes inspired by the Boost library.

Proposal

This paper proposes an iterator facade class template for the standard library, useful by any programmer (novice, experienced, or expert) wishing to easily create a conforming iterator. The proposal uses C++11/14/17 with concepts^[3] and ranges^[4] to allow straightforward specification and implementation, and to ensure that the generated iterator is actually conforming. The proposal breaks no existing code and breaks no existing ABI's. Two open-source implementations with test suites are available on GitHub^[5]^[6].

The proposal is suitable for C++17 if C++17 includes concepts and ranges. Otherwise, the proposal is suitable for the Ranges TS. No other core language or library changes are required.

History

A 2004 proposal^[7] based on Boost `iterator_facade` failed because it depended on an iterator update proposal^[8] that failed because without concepts the language was not rich enough to express the necessary iterator requirements.

Example

```
class cursor
{
    using string = std::string;
    const string* str_;    // nullptr if uninitialized
    string::size_type begin_; // end iterator: begin_ == string::npos
    string word_;
    static constexpr const char* alpha =
        "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";
public:
    cursor() noexcept
        : str_(nullptr), begin_(string::npos), word_() {}
    cursor(const string& str)
        : str_(&str), begin_(0), word_() { next(); }
    const string& read() const noexcept {
        assert(str_);           // fails if uninitialized
        assert(begin_ != string::npos); // fails on dereference end
        return word_;
    }
    bool equal(const cursor& rhs) const noexcept
        { return begin_ == rhs.begin_; }
    void next() {
        assert(str_);           // fails if uninitialized
        assert(begin_ != string::npos); // fails on increment past end
        if ((begin_ += word_.size()) != string::npos
            && (begin_ = str_>find_first_of(alpha, begin_)) != string::npos)
            word_.assign(*str_, begin_,
                str_>find_first_not_of(alpha, begin_) - begin_);
        }
};

using word_iterator = ranges::basic_iterator<cursor>;
```

The five member functions in class `cursor` result in generation of approximately eleven class `word_iterator` functions, as well supply in several useful iterator type-traits.

For a sample program that uses `word_iterator`, [see Basic Iterators, below](#).

Design Decisions

Use Boost `iterator_facade` as a role model

This ensures that the proposal represents existing practice in widespread use.

Base design on Concepts and Ranges

This eliminates the difficulties with specification that bedeviled the 2004 proposal. It allows a

concepts based interface specification that ensures the resulting iterator is actually conforming. It improves error message quality when the user makes a mistake. Using concepts based overloading eliminates the need for the implementation to perform complex template meta-programming.

Use Cursor mixins to supply details

Cursor mixins have proven themselves useful time and again. That said, it's a curiously indirect way of defining an iterator's interface. The alternative of inheriting directly from the Cursor leads to interface pollution. Cursors are implementation details and they should stay hidden.

Proposed wording

Editorial comments are shown in italics with a light grey background.

Proposed wording is relative to the Working Draft, C++ Extensions for Ranges

Namespace qualification in the form `ranges::` is used as a placeholder pending a decision as to the final target for this proposal. If the target is the Ranges TS, `ranges::` will be replaced editorially by `std::experimental::ranges::`. If the target is the standard itself, `ranges::` will be replaced by `std::`, assuming the standard library itself does not introduce namespace versioning.

Add the following proposed wording as a new Iterator adaptor at the end of 24.8 Iterator adaptors [iterators.predef]:

24.8.9 Basic Iterators [iterators.basic]

Class template `basic_iterator` ([iterator.basic_iterator](#)) is an iterator adaptor that iterates over a sequence provided by a cursor type. [Note: `basic_iterator` eases creation of conforming iterators because cursors are simpler to create than iterators. -- end note] Cursors are implementation details of a `basic_iterator` instantiation that are encapsulated as mixins so that they are hidden from users of the instantiation.

[Example:

```
class cursor
{
    using string = std::string;
    const string* str_; // nullptr if uninitialized
    string::size_type begin_; // end iterator: begin_ == string::npos
    string word_;
    static constexpr const char* alpha =
        "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";
public:
    cursor() noexcept
        : str_(nullptr), begin_(string::npos), word_() {}
    cursor(const string& str)
        : str_(&str), begin_(0), word_() { next(); }
    const string& read() const noexcept {
        assert(str_); // fails if uninitialized
        assert(begin_ != string::npos); // fails on dereference end
        return word_;
    }
    bool equal(const cursor& rhs) const noexcept
        { return begin_ == rhs.begin_; }
    void next() {
        assert(str_); // fails if uninitialized
        assert(begin_ != string::npos); // fails on increment past end
        if ((begin_ += word_.size()) != string::npos
            && (begin_ = str_->find_first_of(alpha, begin_)) != string::npos)
            word_.assign(*str_, begin_,
                str_>find_first_not_of(alpha, begin_) - begin_);
    }
};

using word_iterator = ranges::basic_iterator<cursor>;
```

Program using `word_iterator`:

```
int main()
{
    std::string s
        ("now is 2016 the time when\nall good programmers should-party.");

    for (word_iterator it(s); it != word_iterator(); ++it)
        std::cout << *it << " (" << it->size() << ")n";
}
```

When executed, the output is:

```
now (3)
is (2)
the (3)
time (4)
when (4)
all (3)
good (4)
programmers (11)
should (6)
party (5)
```

-- end example]

A cursor `C` may extend the interface of `basic_iterator<C>` by defining a nested mixin type `C::mixin`. In that way, the author of a cursor can non-intrusively add members and constructors to `basic_iterator`. If a cursor does not define a nested mixin type, a default mixin type will be provided.

[Note: By publicly inheriting from a mixin type parameterized by the cursor type, `basic_iterator` gets access to the cursor object, and, optionally, additional members such as constructors. -- end note]

This sub-clause has three major sub-sections:

- [Namespace cursor](#) describes cursor types.
- [Class template basic_mixin](#) describes mixin types.
- [Class template basic_iterator](#) describes `basic_iterator`.

24.8.9.1 Cursors [cursor.intro]

Namespace `cursor` provides a scope for the type traits, concepts, and other traits needed to describe cursor types.

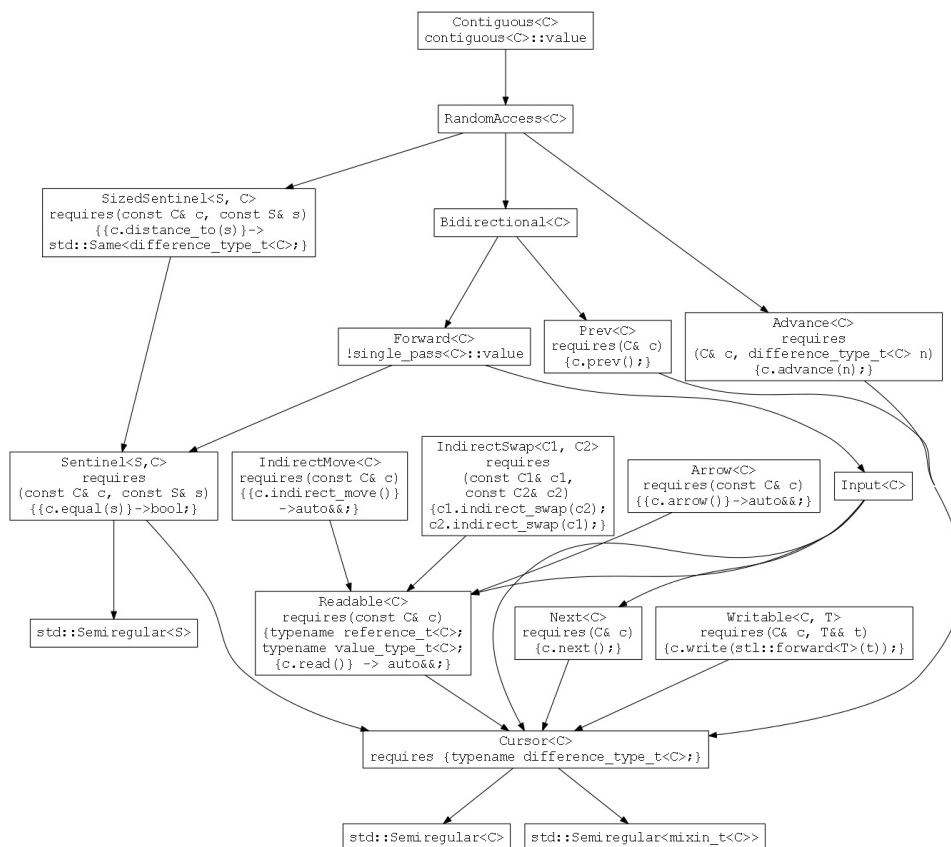
Which cursor concepts are satisfied by a user-supplied cursor type is determined by its members. The relationship between a cursor type's member, the cursor concept that requires it, and a summary of the cursor concept's requirement for the member are shown by the following table. The table is informational; actual requirements are specified by the concept descriptions that follow.

Member name	Required by Concept	Cursor member requirement summary
read	Readable<C>	requires(const C& c) { {c.read()} -> auto&&; typename reference_t<C>; typename value_type_t<C>; }
arrow	Arrow<C>	requires(const C& c) { {c.arrow()} -> auto&&; }
next	Next<C>	requires(C& c) {c.next();}
prev	Prev<C>	requires(C& c) {c.prev();}
indirect_move	IndirectMove<C>	requires(const C& c) {c.indirect_move();}
indirect_swap	IndirectSwap<C1, C2>	requires(const C1& c1, const C2& c2) {c1.indirect_swap(c2); c2.indirect_swap(c1);}
advance	Advance<C>	requires(C& c, difference_type_t<C> n) {c.advance(n);}
write	Writable<C, T>	requires(C& c, T&& t) {c.write(range::forward<T>(t));}
distance_to	SizedSentinel<S, C>	requires(const C& c, const S& s) { {c.distance_to(s)} -> Same<difference_type_t<C>; }
equal	Sentinel<S, C>	requires(const C& c, const S& s) { {c.equal(s)}->bool; }
single_pass::value	Forward<C>	bool single_pass::value default: =false
contiguous::value	Contiguous<C>	is_reference<reference_t<C>>::value default: =false
		type mixin

mixin	Cursor<C>	default: =basic_mixin<C>
value_type	Readable<C>	type value_type default: =decay_t<reference_t<C>>

Cursor members shown with defaults are only required if the default is not appropriate.

The following figure shows the relationship between cursor related concepts. The figure is informational; actual relationships are specified by the concepts descriptions that follow.



24.8.9.1.1 Namespace cursor synopsis [cursor.synopsis]

```

namespace std {
namespace experimental {
namespace ranges {
inline namespace v1 {
namespace cursor {

// Cursor traits

// single_pass trait
template <class> constexpr bool single_pass = false;
template <class C>
requires requires {
    typename C::single_pass;
    requires bool(C::single_pass::value);
}
constexpr bool single_pass = true;

// contiguous trait
template <class> constexpr bool contiguous = false;
template <class C>
requires requires {
    typename C::contiguous;
    requires bool(C::contiguous::value);
}
constexpr bool contiguous = true;

// category trait
template <class>
struct category {};
template <Input C>
struct category<C> { using type = input_iterator_tag; };

```

```

template <Forward C>
    struct category<C> { using type = forward_iterator_tag; };
template <Bidirectional C>
    struct category<C> { using type = bidirectional_iterator_tag; };
template <RandomAccess C>
    struct category<C> { using type = random_access_iterator_tag; };
template <Contiguous C>
    struct category<C> { using type = ext::contiguous_iterator_tag; };
template <class C>
    using category_t = typename category<C>::type;

// types
template <class C>
    using mixin_t = see below;
template <class C>
    requires see below
    using value_type_t = see below;
template <class C>
    using difference_type_t = see below;
template <class C>
    requires
        requires(const C& c) {{c.read()} -> auto&&};
    using reference_t = see below;

// concepts
template <class C>
    concept bool Cursor();
template <class C>
    concept bool Readable();
template <class C>
    concept bool Arrow();
template <class C, class T>
    concept bool Writable();
template <class S, class C>
    concept bool Sentinel();
template <class S, class C>
    concept bool SizedSentinel();
template <class C>
    concept bool Next();
template <class C>
    concept bool Prev();
template <class C>
    concept bool Advance();
template <class C>
    concept bool IndirectMove();
template <class C, class O>
    concept bool IndirectSwap();
template <class C>
    concept bool Input();
template <class C>
    concept bool Forward();
template <class C>
    concept bool Bidirectional();
template <class C>
    concept bool RandomAccess();
template <class C>
    concept bool Contiguous();
}}}}}

```

24.8.9.1.2 Cursor traits [cursor.traits]

24.8.9.1.2.1 single_pass [cursor.single]

```

template <class> constexpr bool single_pass = false;
template <class C>
    requires requires {
        typename C::single_pass;
        requires bool(C::single_pass::value);
    }
constexpr bool single_pass = true;

```

using single_pass = std::true_type; is defined by a cursor to specify to basic_iterator that the cursor does not satisfy the cursor::ForwardIterator concept despite satisfying the cursor::InputIterator and cursor::EqualityComparable concepts.

24.8.9.1.2.2 contiguous [cursor.contig]

```
template <class> constexpr bool contiguous = false;
template <class C>
requires requires {
    typename C::contiguous;
    requires bool(C::contiguous::value);
}
constexpr bool contiguous = true;
```

using contiguous = std::true_type; is defined by a cursor to specify to basic_iterator that the cursor satisfies the cursor::Contiguous concept if it also satisfies the cursor::RandomAccess concept.

24.8.9.1.2.3 Types [cursor.types]

These type traits are used in cursor concepts, traits, and in class basic_iterator to access types defined by cursors or deduced from the presence of cursor functions.

```
template <class C>
using mixin_t = see below; // used by concepts, etc
```

Type mixin_t is defined as C::mixin if type C::mixin is defined. Otherwise it is defined as basic_mixin<C>.

```
template <class C>
requires see below
using value_type_t = see below; // used by concepts, etc.
```

The requires clause is satisfied if and only if Same<deduced_value_t<C>::type, decay_t<deduced_value_t<C>::type>>() would be satisfied.

Type value_type_t is defined as deduced_value_t<C>::type.

Remarks: template <class C> deduced_value_t; is an exposition only type defined as:

- struct value_type<C> {using type = typename C::value_type;}; if C has a member value_type,
- Otherwise struct value_type<C> {using type = decay_t<reference_t<C>>}; if C does not have a member value_type and satisfies a requirement for reference_t<C>.
- Otherwise struct deduced_value_t {};

```
template <class C>
using difference_type_t = see below; // used by concepts, etc.
```

Type difference_type_t is defined as:

- C::difference_type if C has a member difference_type,
- Otherwise decltype(declval<const C&>().distance_to(declval<const C&>())) if C has such a distance_to member function,
- Otherwise std::ptrdiff_t.

```
template <class C>
requires
    requires(const C& c) {c.read() -> auto&&};
using reference_t = see below; // used by traits, etc.
```

Type reference_t is defined as decltype(declval<const C&>().read()).

24.8.9.1.3 Concepts [cursor.concepts]

```
template <class C>
concept bool Cursor();
```

Returns: Semiregular<remove_cv_t<C>>()
&& Semiregular<mixin_t<remove_cv_t<C>>>()
&& requires {typename difference_type_t<C>;}.

```
template <class C>
```

```
concept bool Readable();
```

Returns: Cursor<C>() && requires(const C& c) {
 {c.read()} -> auto&&;
 typename reference_t<C>;
 typename value_type_t<C>;
}.

```
template <class C>  
concept bool Arrow();
```

Returns: Readable<C>()
&& requires(const C& c) {{c.arrow()} -> auto&&;}.

```
template <class C, class T>  
concept bool Writable();
```

Returns: Cursor<C>()
&& requires(C& c, T&& t) {c.write(ranges::forward<T>(t));}.

Remarks: Not required to be equality-preserving.

```
template <class S, class C>  
concept bool Sentinel();
```

Returns: Cursor<C>() && Semiregular<S>()
&& requires(const C& c, const S& s) {{c.equal(s)} -> bool;}.

```
template <class S, class C>  
concept bool SizedSentinel();
```

Returns: Sentinel<S, C>() && requires(const C& c, const S& s)
{{c.distance_to(s)} -> Same<difference_type_t<C>;}.

```
template <class C>  
concept bool Next();
```

Returns: Cursor<C>() && requires(C& c) {c.next();}.

```
template <class C>  
concept bool Prev();
```

Returns: Cursor<C>() && requires(C& c) {c.prev();}.

```
template <class C>  
concept bool Advance();
```

Returns: Cursor<C>()
&& requires(C& c, difference_type_t<C> n) {c.advance(n);}.

```
template <class C>  
concept bool IndirectMove();
```

Returns: Readable<C>()
&& requires(const C& c) {c.indirect_move()} -> auto&&;}.

```
template <class C1, class C2>  
concept bool IndirectSwap();
```

Returns: Readable<C1>() && Readable<C2>()
&& requires(const C1& c1, const C2& c2)
{c1.indirect_swap(c2); c2.indirect_swap(c1);}.

Axiom: If c1.read() == x and c2.read() == y then after either c1.indirect_swap(c2)
or c2.indirect_swap(c1), c1.read() == y and c2.read() == x. No diagnostic
required.

```
template <class C>  
concept bool Input();
```

Returns: Readable<C>() && Next<C>().

```
template <class C>  
concept bool Forward();
```

Returns: Input<C>() && Sentinel<C, C>() && !single_pass<C>.


```
template <class C>
concept bool Bidirectional();
```

Returns: Forward<C>() && Prev<C>().

```
template <class C>
concept bool RandomAccess();
```

Returns: Bidirectional<C>()
&& Advance<C>() && SizedSentinel<C, C>().

```
template <class C>
concept bool Contiguous();
```

Returns: RandomAccess<C>() && contiguous<C>
&& is_reference<reference_t<C>>::value.

Add to 24.6, Header <experimental/ranges/iterator> synopsis [iterator.synopsis]:

```
// basic_mixin
template <Destructible T>
class basic_mixin;
```

Continue to add to Basic Iterators [iterators.basic]:

24.8.9.2 Class template basic_mixin [iterator.mixin]

Class template basic_mixin describes a mixin type.

Class basic_mixin inherits from template parameter T or from an implementation-supplied base class that inherits from template parameter T.

[Note: Permitting an implementation-supplied base class gives an implementation latitude to perform empty base optimization if it so chooses. -- end note]

```
template <Destructible T>
class basic_mixin : protected see below {
public:
    // constructors
    constexpr basic_mixin()
        noexcept(is_nothrow_default_constructible<T>::value)
        requires DefaultConstructible<T>();
    constexpr basic_mixin(const T& t)
        noexcept(is_nothrow_copy_constructible<T>::value)
        requires CopyConstructible<T>();
    constexpr basic_mixin(T&& t)
        noexcept(is_nothrow_move_constructible<T>::value)
        requires MoveConstructible<T>();

    // T object access
    constexpr T& get() & noexcept;
    constexpr const T& get() const& noexcept;
    constexpr T&& get() && noexcept;
    constexpr const T&& get() const&& noexcept;
};
```

24.8.9.2.1 Constructors [mixin.cons]

```
constexpr basic_mixin()
    noexcept(is_nothrow_default_constructible<T>::value)
    requires DefaultConstructible<T>();
```

Effects: Default constructs an object of class basic_mixin.

Postconditions: get() returns a reference to a default constructed object or sub-object of type T.

```
constexpr basic_mixin(const T& t)
    noexcept(is_nothrow_copy_constructible<T>::value)
    requires CopyConstructible<T>();
```

Effects: Copy constructs an object of type basic_mixin.

Postconditions: get() returns a reference to a copy constructed object or sub-object of type T with same state as t.

```
constexpr basic_mixin(T&& t)
noexcept(is_nothrow_move_constructible<T>::value)
requires MoveConstructible<T>();
```

Effects: Move constructs an object of type `basic_mixin`.

Postconditions: `get()` returns a reference to a move constructed object or sub-object of type `T` with the state of `ranges::move(t)`.

24.8.9.2.2 T object access [mixin.access]

The `get` member functions are permitted return a reference to either an object or a sub-object. [Note: This allows an implementation to return a reference to either a data member of type `T` or `*this` respectively, depending on implementation details. -- end note]

```
constexpr T& get() & noexcept;
constexpr const T& get() const& noexcept;
```

Returns: A reference to the object or sub-object of type `T` created when `*this` was constructed.

```
constexpr T&& get() && noexcept;
constexpr const T&& get() const&& noexcept;
```

Returns: `std::move(x)`, where `x` is a reference to the object or sub-object of type `T` created when `*this` was constructed..

Continue to add to Basic Iterators [iterators.basic]:

24.8.9.3 Class template `basic_iterator` [iterator.basic_iterator]

Class template `basic_iterator` describes an iterator over a sequence. A `basic_iterator` instantiation satisfies concept `ranges::Iterator`. Which of the other iterator concepts will be satisfied is determined by which cursor concepts are satisfied by the `basic_iterator` template parameter `C`.

24.8.9.3.1 Synopsis [basic_iterator.synopsis]

```
namespace std {
namespace experimental {
namespace ranges {
inline namespace v1 {

template <Cursor C>
class basic_iterator
: public mixin_t<C>
{
private:
// all private members for exposition only
using mixin = mixin_t<C>;
using mixin::get;

using assoc_t = see below;
using typename assoc_t::postfix_increment_result_t;
using typename assoc_t::reference_t;
using typename assoc_t::const_reference_t;

using difference_type = cursor::difference_type_t<C>;
public:

// constructors, assignments, moves, swaps
basic_iterator() = default;

using mixin::mixin;

template <ConvertibleTo<C> O>
constexpr basic_iterator(basic_iterator<O>&& that)
noexcept(is_nothrow_constructible<mixin, O&&::value>);
template <ConvertibleTo<C> O>
constexpr basic_iterator(const basic_iterator<O>& that)
noexcept(is_nothrow_constructible<mixin, const O&&::value>);

template <ConvertibleTo<C> O>
```

```

constexpr basic_iterator& operator=(basic_iterator<O>&& that) &
    noexcept(is_nothrow_assignable<C&, O&&>::value);
template <ConvertibleTo<C> O>
constexpr basic_iterator& operator=(const basic_iterator<O>& that) &
    noexcept(is_nothrow_assignable<C&, const O&>::value);

template <class T>
requires
    !Same<decay_t<T>, basic_iterator>() &&
    !cursor::Next<C>() &&
    cursor::Writable<C, T>()
constexpr basic_iterator& operator=(T&& t) &
    noexcept(noexcept(declval<C&>().write(static_cast<T&&>(t))));

friend constexpr decltype(auto) iter_move(const basic_iterator& i)
    noexcept(noexcept(i.get().indirect_move()))
    requires cursor::IndirectMove<C>();

template <class O>
requires cursor::IndirectSwap<C, O>()
friend constexpr void iter_swap(
    const basic_iterator& x, const basic_iterator<O>& y)
    noexcept(noexcept((void)x.indirect_swap(y)));

// dereferences
constexpr decltype(auto) operator*() const
    noexcept(noexcept(declval<const C&>().read()))
    requires cursor::Readable<C>() && !detail::is_writable<C>;
constexpr decltype(auto) operator*()
    noexcept(noexcept(reference_t{declval<mixin&>().get()}))
    requires cursor::Next<C>() && detail::is_writable<C>;
constexpr decltype(auto) operator*() const
    noexcept(noexcept(
        const_reference_t{declval<const mixin&>().get()}))
    requires cursor::Next<C>() && detail::is_writable<C>;
constexpr basic_iterator& operator*() noexcept
    requires !cursor::Next<C>();

// operator->: "Manual" deduction override,
constexpr decltype(auto) operator->() const
    noexcept(noexcept(declval<const C&>().arrow()))
    requires cursor::Arrow<C>();
// operator->: Otherwise, if reference_t is an lvalue reference,
constexpr auto operator->() const
    noexcept(noexcept(*declval<const basic_iterator&>()))
    requires cursor::Readable<C>() && !cursor::Arrow<C>()
    && is_lvalue_reference<const_reference_t>::value;
// operator->: Otherwise, deduce if needed
constexpr auto operator->() const
    noexcept(is_nothrow_move_constructible<
        detail::operator_arrow_proxy<basic_iterator>>::value &&
        noexcept(detail::operator_arrow_proxy<basic_iterator>{
            *declval<const basic_iterator&>()}))
    requires cursor::Readable<C>() && !cursor::Arrow<C>()
    && !is_reference<const_reference_t>::value;

// modifiers
constexpr basic_iterator& operator++() & noexcept;
constexpr basic_iterator& operator++() &
    noexcept(noexcept(declval<C&>().next()))
    requires cursor::Next<C>();

constexpr postfix_increment_result_t operator++(int) &
    noexcept(is_nothrow_constructible<postfix_increment_result_t,
        basic_iterator&>::value
        && is_nothrow_move_constructible<postfix_increment_result_t>::value
        && noexcept(++declval<basic_iterator&>()));

constexpr basic_iterator& operator--() &
    noexcept(noexcept(declval<C&>().prev()))
    requires cursor::Bidirectional<C>();

```

```

constexpr basic_iterator operator--(int) &
    noexcept(is_nothrow_copy_constructible<basic_iterator>::value &&
              is_nothrow_move_constructible<basic_iterator>::value &&
              noexcept(--declval<basic_iterator>&()))
    requires cursor::Bidirectional<C>();

constexpr basic_iterator& operator+=(difference_type n) &
    noexcept(noexcept(declval<C>&().advance(n)))
    requires cursor::RandomAccess<C>();

constexpr basic_iterator& operator-=(difference_type n) &
    noexcept(noexcept(declval<C>&().advance(-n)))
    requires cursor::RandomAccess<C>();

constexpr decltype(auto) operator[](difference_type n) const
    noexcept(noexcept(*(declval<basic_iterator>&() + n)))
    requires cursor::RandomAccess<C>();

// non-template type-symmetric ops to enable implicit conversions
friend constexpr difference_type operator-(
    const basic_iterator& x, const basic_iterator& y)
    noexcept(noexcept(y.get().distance_to(x.get())))
    requires cursor::SizedSentinel<C, C>();
friend constexpr bool operator==(
    const basic_iterator& x, const basic_iterator& y)
    noexcept(noexcept(x.get().equal(y.get())))
    requires cursor::Sentinel<C, C>();
friend constexpr bool operator!=(
    const basic_iterator& x, const basic_iterator& y)
    noexcept(noexcept(!(x == y)))
    requires cursor::Sentinel<C, C>();
friend constexpr bool operator<(
    const basic_iterator& x, const basic_iterator& y)
    noexcept(noexcept(x - y))
    requires cursor::SizedSentinel<C, C>();
friend constexpr bool operator>(
    const basic_iterator& x, const basic_iterator& y)
    noexcept(noexcept(x - y))
    requires cursor::SizedSentinel<C, C>();
friend constexpr bool operator<=(
    const basic_iterator& x, const basic_iterator& y)
    noexcept(noexcept(x - y))
    requires cursor::SizedSentinel<C, C>();
friend constexpr bool operator>=(
    const basic_iterator& x, const basic_iterator& y)
    noexcept(noexcept(x - y))
    requires cursor::SizedSentinel<C, C>();
};

// basic_iterator nonmember traits
template <class C>
    struct difference_type<basic_iterator<C>>
    { using type = cursor::difference_type_t<C>; };
template <cursor::Input C>
    struct iterator_category<basic_iterator<C>>
    { using type = cursor::category_t<C>; };
template <cursor::Input C>
    struct value_type<basic_iterator<C>>
    { using type = cursor::value_type_t<C>; };

// basic_iterator nonmember functions
template <_InstanceOf<basic_iterator> BI>
    constexpr decltype(auto) get_cursor(BI&& i)
        noexcept(noexcept(std::forward<BI>(i).get()));

template <class C>
    constexpr basic_iterator<C> operator+(
        const basic_iterator<C>& i, cursor::difference_type_t<C> n)
        noexcept(is_nothrow_copy_constructible<basic_iterator<C>>::value &&
                  is_nothrow_move_constructible<basic_iterator<C>>::value &&

```

```

    noexcept(declval<basic_iterator<C>&>() += n))
requires cursor::RandomAccess<C>();
template <class C>
constexpr basic_iterator<C> operator+(
    cursor::difference_type_t<C> n, const basic_iterator<C>& i)
noexcept(noexcept(i + n))
requires cursor::RandomAccess<C>();

template <class C>
constexpr basic_iterator<C> operator-(
    const basic_iterator<C>& i, cursor::difference_type_t<C> n)
noexcept(noexcept(i - n))
requires cursor::RandomAccess<C>();
template <class C1, class C2>
requires cursor::SizedSentinel<C1, C2>()
constexpr cursor::difference_type_t<C2> operator-(
    const basic_iterator<C1>& lhs, const basic_iterator<C2>& rhs)
noexcept(noexcept(
    ranges::get_cursor(rhs).distance_to(ranges::get_cursor(lhs))));
template <class C, class S>
requires cursor::SizedSentinel<S, C>()
constexpr difference_type_t<C> operator-(
    const S& lhs, const basic_iterator<C>& rhs)
noexcept(noexcept(ranges::get_cursor(rhs).distance_to(lhs)));
template <class C, class S>
requires cursor::SizedSentinel<S, C>()
constexpr difference_type_t<C> operator-(
    const basic_iterator<C>& lhs, const S& rhs)
noexcept(noexcept(-(rhs - lhs)));

template <class C1, class C2>
requires cursor::Sentinel<C2, C1>()
constexpr bool operator==(
    const basic_iterator<C1>& lhs, const basic_iterator<C2>& rhs)
noexcept(noexcept(ranges::get_cursor(lhs).equal(ranges::get_cursor(rhs))));
template <class C, class S>
requires cursor::Sentinel<S, C>()
constexpr bool operator==(
    const basic_iterator<C>& lhs, const S& rhs)
noexcept(noexcept(ranges::get_cursor(lhs).equal(rhs)));
template <class C, class S>
requires cursor::Sentinel<S, C>()
constexpr bool operator==(
    const S& lhs, const basic_iterator<C>& rhs)
noexcept(noexcept(rhs == lhs));

template <class C1, class C2>
requires cursor::Sentinel<C2, C1>()
constexpr bool operator!=(
    const basic_iterator<C1>& lhs, const basic_iterator<C2>& rhs)
noexcept(noexcept(!(lhs == rhs)));
template <class C, class S>
requires cursor::Sentinel<S, C>()
constexpr bool operator!=(
    const basic_iterator<C>& lhs, const S& rhs)
noexcept(noexcept(!ranges::get_cursor(lhs).equal(rhs)));
template <class C, class S>
requires cursor::Sentinel<S, C>()
constexpr bool operator!=(
    const S& lhs, const basic_iterator<C>& rhs)
noexcept(noexcept(!ranges::get_cursor(rhs).equal(lhs)));

template <class C1, class C2>
requires cursor::SizedSentinel<C1, C2>()
constexpr bool operator<(
    const basic_iterator<C1>& lhs, const basic_iterator<C2>& rhs)
noexcept(noexcept(lhs - rhs < 0));

template <class C1, class C2>
requires cursor::SizedSentinel<C1, C2>()
constexpr bool operator>(

```

```

    const basic_iterator<C1>& lhs, const basic_iterator<C2>& rhs)
    noexcept(noexcept(lhs - rhs > 0));

template <class C1, class C2>
    requires cursor::SizedSentinel<C1, C2>()
constexpr bool operator<=(
    const basic_iterator<C1>& lhs, const basic_iterator<C2>& rhs)
    noexcept(noexcept(lhs - rhs <= 0));

template <class C1, class C2>
    requires cursor::SizedSentinel<C1, C2>()
constexpr bool operator>=(
    const basic_iterator<C1>& lhs, const basic_iterator<C2>& rhs)
    noexcept(noexcept(lhs - rhs >= 0));
}}}
```

24.8.9.3.2 Requirements [basic_iterator.require]

Private members of class `basic_iterator` are for exposition only (C++17 17.5.2.3 Private members [objects.within.classes]).

```

using mixin = mixin_t<C>;
using mixin::get;
```

Remarks: Provides access to the `get` functions described in [mixin object access \[mixin.object\]](#).

```

using assoc_t = see below;
```

Remarks: `assoc_t` is an alias for an unspecified type containing several implementation-supplied types.

```

using typename assoc_t::postfix_increment_result_t;
```

Remarks: If `cursor::Next<C>` is satisfied, `postfix_increment_result_t` shall satisfy the requirements imposed by [operator++\(int\) below](#) and concept `WeaklyIncrementable<basic_iterator<C>>` (RangesTS[iterators.weaklyincrementable]). If `cursor::Next<C>` is not satisfied, `postfix_increment_result_t` has no requirements.

```

using typename assoc_t::reference_t;
```

TBS

```

using typename assoc_t::const_reference_t;
```

TBS

```

using difference_type = cursor::difference_type_t<C>;
```

TBS

24.8.9.3.3 Constructors, assignments, and moves [basic_iterator.cons]

```

template <ConvertibleTo<C> O>
constexpr basic_iterator(const basic_iterator<O>& that)
    noexcept(is_nothrow_constructible<mixin, const O&>::value);
```

Effects: Constructs an object of class `basic_iterator`.

Postconditions: The state of `*this` is the same as that.

```

template <ConvertibleTo<C> O>
constexpr basic_iterator(basic_iterator<O>&& that)
    noexcept(is_nothrow_constructible<mixin, O&&>::value);
```

Effects: Constructs an object of class `basic_iterator`.

Postconditions: The state of `*this` is the same as the initial state of `that`. `that` is in a valid but unspecified state.

```

template <ConvertibleTo<C> O>
constexpr basic_iterator& operator=(const basic_iterator<O>& that) &
    noexcept(is_nothrow_assignable<C&, const O&>::value);
```

Effects: If `*this` and `that` are not the same object, copy assigns `that` to `*this`.

Otherwise no effect.

Returns: *this.

Postconditions: The state of *this is the same as that.

```
template <ConvertibleTo<C> O>
constexpr basic_iterator& operator=(basic_iterator<O>&& that) &
noexcept(is_nothrow_assignable<C&, O&&>::value);
```

Effects: Move assigns that to *this.

Returns: *this.

Postconditions: The state of *this is the same as the initial state of that. that is left in a valid but unspecified state.

```
template <class T>
requires
!Same<decay_t<T>, basic_iterator>() &&
!cursor::Next<C>() &&
cursor::Writable<C, T>()
constexpr basic_iterator& operator=(T&& t) &
noexcept(noexcept(declval<C&>().write(static_cast<T&&>(t))));
```

Effects: Move assigns t to *this.

Returns: *this.

Postconditions: The state of *this is the same as the initial state of t. t is left in a valid but unspecified state.

```
friend constexpr decltype(auto) iter_move(const basic_iterator& i)
noexcept(noexcept(i.get().indirect_move()))
requires cursor::IndirectMove<C>();
```

TBS

```
template <class O>
requires cursor::IndirectSwap<C, O>()
friend constexpr void iter_swap(
const basic_iterator& x, const basic_iterator<O>& y)
noexcept(noexcept((void)x.indirect_swap(y)));
```

TBS

24.8.9.3.4 Dereferences [basic_iterator.deref]

```
constexpr decltype(auto) operator*() const
noexcept(noexcept(declval<const C&>().read()));
```

Returns: get().read().

Remarks: get().read() requires requires(C& c) { c.read(); }.

```
constexpr decltype(auto) operator*() noexcept
requires cursor::is_writable<C>;
```

Returns: reference_t{get()}.

```
constexpr decltype(auto) operator*() const noexcept
requires cursor::is_writable<C>;
```

Returns: const_reference_t{get()}.

```
constexpr decltype(auto) operator->() const
noexcept(noexcept(declval<const C&>().arrow()))
requires cursor::Arrow<const C>();
```

Returns: get().arrow().

24.8.9.3.5 Modifiers [basic_iterator.mods]

```
constexpr basic_iterator& operator++() & noexcept;
```

Effects: None.

Returns: *this.

[Note: This overload is only selected if the following overload is not selected. -- end note]

```
constexpr basic_iterator& operator++() &  
noexcept(noexcept(declval<C&>().next()))  
requires cursor::Next<C>();
```

Effects: get().next().

Returns: *this.

```
constexpr basic_iterator& operator++(int) & noexcept;
```

Effects: None.

Returns: *this.

[Note: This overload is only selected if the following overload is not selected. -- end note]

```
constexpr postfix_increment_result_t operator++(int) &  
noexcept(is_nothrow_constructible<postfix_increment_result_t,  
basic_iterator&>::value  
&& is_nothrow_move_constructible<postfix_increment_result_t>::value &&  
noexcept(++declval<basic_iterator&>()))  
requires cursor::Next<C>();
```

Effects:

postfix_increment_result_t tmp(*this);

++*this;

return tmp;

```
constexpr basic_iterator& operator--() &  
noexcept(noexcept(declval<C&>().prev()))  
requires cursor::Bidirectional<C>();
```

Effects: get().prev();.

Returns: *this.

```
constexpr basic_iterator operator--(int) &  
noexcept(is_nothrow_copy_constructible<basic_iterator>::value &&  
is_nothrow_move_constructible<basic_iterator>::value &&  
noexcept(--declval<basic_iterator&>()))  
requires cursor::Bidirectional<C>();
```

Effects:

auto tmp = *this;

--*this;

return tmp;

```
constexpr basic_iterator& operator+=(difference_type n) &  
noexcept(noexcept(declval<C&>().advance(n)))  
requires cursor::RandomAccess<C>();
```

Effects: get().advance(n).

Returns: *this.

```
constexpr basic_iterator& operator-=(difference_type n) &  
noexcept(noexcept(declval<C&>().advance(-n)))  
requires cursor::RandomAccess<C>();
```

Effects: get().advance(-n).

Returns: *this.

```
friend constexpr basic_iterator  
operator+(const basic_iterator& i, difference_type n)  
noexcept(is_nothrow_copy_constructible<basic_iterator>::value &&  
is_nothrow_move_constructible<basic_iterator>::value &&  
noexcept(declval<C&>().advance(n)))  
requires cursor::RandomAccess<C>();
```

Effects:

auto tmp = i;


```
tmp.get().advance(n);  
return tmp;
```

```
friend constexpr basic_iterator  
operator+(difference_type n, const basic_iterator& i)  
noexcept(noexcept(i + n))  
requires cursor::RandomAccess<C>();
```

Returns: i + n.

```
friend constexpr basic_iterator  
operator-(const basic_iterator& i, difference_type n)  
noexcept(noexcept(i - n))  
requires cursor::RandomAccess<C>();
```

Returns: i - n.

```
constexpr decltype(auto) operator[](difference_type n) const  
noexcept(noexcept(*(declval<basic_iterator>() + n)))  
requires cursor::RandomAccess<C>();
```

Returns: *(*this + n).

24.8.9.3.6 basic_iterator nonmember functions [basic_iterator.nonmem]

```
template <class C>  
constexpr C& get_cursor(basic_iterator<C>& i)  
noexcept(noexcept(i.get()))
```

Returns: i.get().

```
template <class C>  
constexpr const C& get_cursor(const basic_iterator<C>& i)  
noexcept(noexcept(i.get()))
```

Returns: i.get().

```
template <class C>  
constexpr C&& get_cursor(basic_iterator<C>&& i)  
noexcept(noexcept(i.get()))
```

Returns: std::move(i.get()).

```
template <class C>  
requires cursor::Sentinel<C, C>()  
constexpr bool operator==(  
const basic_iterator<C>& lhs, const basic_iterator<C>& rhs)  
noexcept(noexcept(  
static_cast<bool>(ranges::get_cursor(lhs).equal(ranges::get_cursor(rhs))))
```

Returns: ranges::get_cursor(lhs).equal(ranges::get_cursor(rhs)).

```
template <class C, class S>  
requires cursor::Sentinel<S, C>()  
constexpr bool operator==(  
const basic_iterator<C>& lhs, const S& rhs)  
noexcept(noexcept(ranges::get_cursor(lhs).equal(rhs)))
```

Returns: ranges::get_cursor(lhs).equal(rhs).

```
template <class C, class S>  
requires cursor::Sentinel<S, C>()  
constexpr bool operator!=(  
const S& lhs, const basic_iterator<C>& rhs)  
noexcept(noexcept(rhs == lhs))
```

Returns: rhs == lhs.

```
template <class C>  
requires cursor::Sentinel<C, C>()  
constexpr bool operator!=(  
const basic_iterator<C>& lhs, const basic_iterator<C>& rhs)  
noexcept(noexcept(!(lhs == rhs)))
```

Returns: !(lhs == rhs).

```
template <class C, class S>
    requires cursor::Sentinel<S, C>()
constexpr bool operator!=(
    const basic_iterator<C>& lhs, const S& rhs)
    noexcept(noexcept(!get_cursor(lhs).equal(rhs)))
```

Returns: !ranges::get_cursor(lhs).equal(rhs).

```
template <class C, class S>
    requires cursor::Sentinel<S, C>()
constexpr bool operator!=(
    const S& lhs, const basic_iterator<C>& rhs)
    noexcept(noexcept(rhs != lhs))
```

Returns: rhs != lhs.

```
template <class C>
    requires cursor::SizedSentinel<C, C>()
constexpr cursor::difference_type_t<C> operator-(
    const basic_iterator<C>& lhs, const basic_iterator<C>& rhs)
    noexcept(noexcept(get_cursor(rhs).distance_to(get_cursor(lhs))))
```

Returns: ranges::get_cursor(rhs).distance_to(ranges::get_cursor(lhs)).

```
template <class C, class S>
    requires cursor::SizedSentinel<S, C>()
constexpr cursor::difference_type_t<C> operator-(
    const S& lhs, const basic_iterator<C>& rhs)
    noexcept(noexcept(get_cursor(rhs).distance_to(lhs)))
```

Returns: get_cursor(rhs).distance_to(lhs).

```
template <class C, class S>
    requires cursor::SizedSentinel<S, C>()
constexpr cursor::difference_type_t<C> operator-(
    const basic_iterator<C>& lhs, const S& rhs)
    noexcept(noexcept(-(rhs - lhs)))
```

Returns: -(rhs - lhs).

```
template <class C>
    requires cursor::SizedSentinel<C, C>()
constexpr bool operator<(
    const basic_iterator<C>& lhs, const basic_iterator<C>& rhs)
    noexcept(noexcept(0 < get_cursor(rhs).distance_to(lhs)))
```

Returns: 0 < ranges::get_cursor(rhs).distance_to(lhs).

```
template <class C>
    requires cursor::SizedSentinel<C, C>()
constexpr bool operator>(
    const basic_iterator<C>& lhs, const basic_iterator<C>& rhs)
    noexcept(noexcept(rhs < lhs))
```

Returns: rhs < lhs.

```
template <class C>
    requires cursor::SizedSentinel<C, C>()
constexpr bool operator<=(
    const basic_iterator<C>& lhs, const basic_iterator<C>& rhs)
    noexcept(noexcept(!(rhs < lhs)))
```

Returns: !(rhs < lhs).

```
template <class C>
    requires cursor::SizedSentinel<C, C>()
constexpr bool operator>=(
    const basic_iterator<C>& lhs, const basic_iterator<C>& rhs)
    noexcept(noexcept(!(lhs < rhs)))
```

Returns: !(lhs < rhs).

Open questions

Is the target C++17 or the the Ranges TS? Deferred until the committee decides if the

Concepts TS and Ranges TS are included in C++17.

Should the class template `basic_mixin` be added to 20 "General utilities" or 24 "Iterators"? The proposed wording adds it to "Iterators", but there is nothing iterator specific in its design or description, and it is generally useful across multiple problem domains. LEWG input would be helpful.

Acknowledgements

Thanks to David Abrahams, Jeremy Siek, and Thomas Witt for Boost `iterator_facade`. Having such a strong role model greatly enriched this proposal.

Thanks to the folks who implemented the Concepts TS for GCC 6.0. Their development snapshots are what allowed this proposal to be implemented and tested.

References

- [1] David Abrahams, Jeremy Siek, Thomas Witt, [Boost Iterator Facade](#), 2003.
 - [2] Chandler Carruth, [LLVM iterator_facade_base iterator.h](#), 2014.
 - [3] Andrew Sutton, [N4553, Working Draft, C++ extensions for Concepts](#), 2015.
 - [4] Eric Niebler, Casey Carter, [N4650, Working Draft, C++ Extensions for Ranges](#), 2015.
 - [5] Eric Niebler, Casey Carter, [Experimental range library for C++11/14/17](#), 2015. Requires a C++14 compiler. Simulates concepts with macros and templates.
 - [6] Casey Carter, Eric Niebler, [An implementation of C++ Extensions for Ranges](#), 2015. Requires a C++14 compiler supporting concepts (e.g. GCC trunk).
 - [7] David Abrahams, Jeremy Siek, Thomas Witt, [N1641, Iterator Facade and Adaptor](#), 2004.
 - [8] David Abrahams, Jeremy Siek, Thomas Witt, [N1640, New Iterator Concepts](#), 2004.
-